

Direct-Gradient Memory Digital MemComputing: Discovering All Five Lagrange Points Across the Solar System Without Prior Knowledge

Dietmar Henrich¹
Massimiliano Di Ventra²

¹Chair of Materials Science and Nanotechnology, Technische Universität Dresden, 01062 Dresden, Germany

²Department of Physics, University of California San Diego, La Jolla, CA 92093, USA

June 19, 2026

Abstract

We present a Digital MemComputing Machine (DMM) that autonomously discovers all five Lagrange equilibrium points (L1–L5) of any two-body system in the solar system *without prior knowledge* of their number or location. The machine receives only the equations of motion in the co-rotating frame and a grid of starting positions; no solution coordinates are provided.

The key theoretical contribution is a *direct-gradient memory rule*: the long-term memory variable for each spatial axis i obeys the ordinary differential equation

$$\dot{w}_L^i = \beta |\partial_i \Omega|,$$

where Ω is the Jacobi effective potential and β a single growth parameter. This removes the short-term exponential average used in earlier formulations, reducing the extended phase space from eight to six dimensions while preserving all convergence guarantees. A complementary *adaptive correction current* evaluates the local y -curvature Ω_{yy} at each step and flips the sign of the y -memory force at collinear saddle points, transforming classically unstable equilibria into stable fixed points of the extended dynamics.

The emulator is validated on 23 two-body pairs spanning Sun–planet and planet–moon systems. All five Lagrange points are found from a generic grid with position errors of order 10^{-5} – 10^{-4} (normalised units). We provide a detailed physical explanation of why DMM overcomes the fundamental obstacles that defeat classical solvers at saddle points, and a systematic comparison with Brent’s method, Newton–Raphson, Nelder–Mead, and gradient descent with momentum. The interactive open-source simulation is available at [15].

1 Introduction

Finding all stationary points of a multi-variable function is one of the central tasks in applied mathematics, yet standard numerical methods remain ill-suited when so-

lutions are saddle points rather than minima. Classical gradient descent is repelled by saddle points; Newton–Raphson requires a good initial guess near every root; root-finding methods such as Brent’s algorithm need the solution to be bracketed in advance; and metaheuristic optimisers such as Nelder–Mead or basin hopping carry no convergence guarantee [8, 9]. The problem is made qualitatively harder when the number and location of solutions are entirely unknown.

MemComputing is a non-von-Neumann computational paradigm in which internal *memory* degrees of freedom retain and exploit past information to solve problems [1]. In a Digital MemComputing Machine (DMM) the memory variables act as *dynamic Lagrange multipliers*: they grow in proportion to the residual, transforming the solution set of the original problem into stable fixed points of an extended dynamical system [2, 3]. Trajectories starting from generic initial conditions are attracted to these fixed points along topologically protected instanton paths, providing convergence without prior knowledge of the solution structure [1].

The restricted three-body problem—two massive primaries in circular orbits and a massless test particle—is a classical benchmark of non-linear celestial mechanics [5]. Its five Lagrange equilibria [6] include three collinear saddle points (L1, L2, L3) and two linearly stable triangular points (L4, L5). Standard approaches locate the collinear points by restricting $y = 0$ and applying root-finding on the resulting one-dimensional equation, exploiting prior knowledge of the symmetry; the triangular points are found analytically. No single unmodified gradient-based method can converge to all five from generic starting positions.

Here we demonstrate that a DMM with a novel *direct-gradient memory rule* overcomes these limitations simultaneously, discovering all five Lagrange points in 23 solar-system two-body pairs from a single two-dimensional grid with no solution-specific knowledge. We derive the physical mechanism responsible for this behaviour and show

why it is fundamentally different from—and complementary to—classical approaches.

2 The Restricted Three-Body Problem

2.1 Effective potential and Lagrange points

Dimensionless co-rotating units are adopted: the distance between the two primaries is unity and the orbital angular frequency $\omega = 1$. The mass ratio is $\mu = M_2/(M_1 + M_2)$, placing the primary at $x_1 = -\mu$ and the secondary at $x_2 = 1 - \mu$. The Jacobi effective potential is

$$\Omega(x, y) = \frac{x^2 + y^2}{2} + \frac{1 - \mu}{r_1} + \frac{\mu}{r_2}, \quad (1)$$

with $r_1 = \sqrt{(x + \mu)^2 + y^2}$ and $r_2 = \sqrt{(x - 1 + \mu)^2 + y^2}$. The equations of motion in the co-rotating frame are

$$\ddot{x} = 2\dot{y} - \partial_x \Omega - \gamma \dot{x}, \quad (2)$$

$$\ddot{y} = -2\dot{x} - \partial_y \Omega - \gamma \dot{y}, \quad (3)$$

where $\pm 2\dot{y}, \pm 2\dot{x}$ are the Coriolis accelerations and γ is a dissipation coefficient introduced for convergence. At a Lagrange point all time derivatives vanish, reducing the condition to $\nabla \Omega = \mathbf{0}$. There are exactly five such points: three collinear points (L1, L2, L3) on the x -axis, and two equilateral triangular points L4/L5 at $(0.5 - \mu, \pm\sqrt{3}/2)$ [5, 6].

2.2 Local curvature and saddle-point instability

The local curvature of Ω in the y -direction is

$$\Omega_{yy} = 1 - \frac{1 - \mu}{r_1^3} + \frac{3(1 - \mu)y^2}{r_1^5} - \frac{\mu}{r_2^3} + \frac{3\mu y^2}{r_2^5}. \quad (4)$$

On the x -axis ($y = 0$), equation (4) reduces to $1 - (1 - \mu)/r_1^3 - \mu/r_2^3$, which is *negative* at all three collinear points. For the Earth–Moon system ($\mu = 0.0121$): L1 gives $\Omega_{yy} = -4.15$, L2 gives -2.19 , and L3 gives -0.011 (weakest saddle). At the triangular points $\Omega_{yy} = +2.25 > 0$.

Crucially, $\text{sign}(\Omega_{yy})$ distinguishes between saddles and stable attractors using *only local information* at the current position, with no prior knowledge of solution locations. Routh’s criterion [7] further requires $\mu < \mu_R \approx 0.03852$ for L4/L5 to be linearly stable in the rotating frame; this condition is evaluated per system.

3 Why Classical Solvers Fail at Saddle Points

3.1 The fundamental obstacle

Standard gradient-descent dynamics follow $\dot{\mathbf{r}} = -\nabla \Omega$. At a collinear Lagrange point with $y > 0$, one has $\partial_y \Omega < 0$

(because $\Omega_{yy} < 0$ means the potential curves *downward* from the x -axis), so $-\partial_y \Omega > 0$: the restoring force points *away* from the equilibrium. Any perturbation in the y -direction is amplified rather than damped. This is the saddle-point repulsion problem: gradient descent cannot converge to a saddle point from any starting position not already exactly on the manifold of attraction [10].

Momentum methods (heavy-ball, Nesterov, Adam) add inertia to gradient descent but do not change the fundamental topology: saddle points remain unstable under first-order gradient dynamics [11]. Second-order methods such as Newton–Raphson can pass through saddle points via the Hessian, but require a starting guess within the basin of each root and diverge without appropriate regularisation when the Hessian is ill-conditioned near the primary bodies. Brent’s method guarantees convergence to a root within a bracket, but the bracket must be supplied explicitly and separately for each root, requiring prior knowledge that collinear L-points are on $y = 0$ and approximate bounds on each. Nelder–Mead minimises a function value and therefore converges only to local minima; the collinear points are saddles of Ω and are invisible to it. The situation is summarised in Table 1.

3.2 Why homotopy continuation is different

Homotopy (path-following) continuation methods [13] can in principle find all roots of a polynomial system, including saddles. They deform a simple start system into the target system along a continuous family and track each solution branch. The approach requires (i) reformulation of the equations as a polynomial or algebraic system, (ii) knowledge of the maximum number of solutions (Bézout bound), and (iii) computational cost that scales combinatorially with problem dimension for dense systems. For problems embedded in higher-dimensional or non-algebraic settings, such as the combinatorial optimisation problems that motivated DMM development, homotopy methods are not applicable.

4 Direct-Gradient Memory DMM

4.1 Clause and variable mapping

The DMM is tasked with satisfying the vector clause

$$C : \quad \nabla \Omega(x, y) = \mathbf{0}, \quad (5)$$

which has five solutions. Table 2 summarises the variable mapping.

4.2 Direct-gradient memory rule

We introduce *per-axis long-term memory variables* w_L^x and w_L^y governed by the first-order ODEs

$$\dot{w}_L^i = \beta |\partial_i \Omega|, \quad i \in \{x, y\}, \quad (6)$$

with initial condition $w_L^i(0) = 1$ and an upper bound $w_L^i \leq w_{\text{cap}}$. The discrete-time update is

$$w_L^i \leftarrow \min(w_L^i + \beta |\partial_i \Omega| \Delta t, w_{\text{cap}}). \quad (7)$$

Table 1: Comparison of methods for finding all five Lagrange points. “Prior knowledge” refers to information about solution structure that must be supplied externally. Function evaluations are for a single L-point (DMM from one grid start); the DMM total covers all five simultaneously. Times measured on an Apple M-series CPU.

Method	Saddles	Prior knowledge	Evals	Time	All 5 Conv.	
Gradient descent	No	None	—	—	No	No
Adam/Nesterov	No	None	—	—	No	No
Newton–Raphson	Yes	1 guess per root	9–34	< 0.1 ms	No	Cond.
Brent’s method	Yes	Bracket + $y=0$	12–18	< 0.1 ms	No	Yes
Nelder–Mead	No	None	150–600	< 1 ms	No	No
Homotopy	Yes	Algebraic form	$\sim 10^3$	~ 1 s	Yes	Yes
DMM (ours)	Yes	None	6×10^3 – 4×10^4	70–400 ms	Yes	Yes

Table 2: DMM variable mapping.

DMM concept	Symbol	Role
Clause	$\nabla\Omega = \mathbf{0}$	L-point condition
Memory variable	w_L^i	Dynamic multiplier
Memory ODE	$\dot{w}_L^i = \beta \partial_i\Omega $	Ratchet rule
Correction current	$\sigma w_L^y \partial_y\Omega$	Sign-adaptive y -force
Instanton path	$\mathbf{r}(t)$	Trajectory to equilibrium

Equation (6) has a transparent closed-form interpretation. Integrating along a trajectory,

$$w_L^i(t) = 1 + \beta \int_0^t |\partial_i\Omega(\mathbf{r}(\tau))| d\tau, \quad (8)$$

so w_L^i accumulates the L^1 arc length of the per-axis gradient along the instanton path. Growth ceases exactly when $|\partial_i\Omega| \rightarrow 0$, i.e. precisely at a solution. This is the *ratchet property*: w_L^i is non-decreasing by construction and saturates only at the target.

Compared with earlier DMM formulations [2, 3] that used a two-stage pipeline (short-term exponential average x_s^i driving w_L^i), the direct rule (6) removes one variable per axis, reducing the extended phase space from eight to six dimensions. It also eliminates the smoothing hyperparameter α , leaving β as the single memory control.

4.3 Adaptive correction current

At each integration step the machine computes Ω_{yy} (equation (4)) from current positions and sets

$$\sigma = \begin{cases} +1 & \Omega_{yy} < 0 \quad (\text{correction current}), \\ -1 & \Omega_{yy} \geq 0 \quad (\text{memory amplification}). \end{cases} \quad (9)$$

No external information about saddle locations is required; σ is determined entirely by local geometry.

4.4 Full DMM equations of motion

Combining the memory variables and the adaptive sign gives

$$\ddot{x} = 2\dot{y} - w_L^x \partial_x\Omega - \gamma\dot{x}, \quad (10)$$

$$\ddot{y} = -2\dot{x} + \sigma w_L^y \partial_y\Omega - \gamma\dot{y}, \quad (11)$$

integrated together with (7) at each step Δt . The extended state vector is $(x, y, \dot{x}, \dot{y}, w_L^x, w_L^y) \in \mathbb{R}^6$. Default parameters are listed in Table 3.

Table 3: Default simulation parameters.

Parameter	Symbol	Value
Time step	Δt	0.01
Memory growth rate	β	0.001
Memory cap	w_{cap}	8.0
Dissipation	γ	0.6
Singularity guard	ε	10^{-9}
Convergence threshold	—	$ \nabla\Omega < 10^{-4}$
Max. steps	—	2×10^5

5 Physical Explanation: Why DMM Succeeds

5.1 Topology change in extended phase space

The key insight is that the extended state space $(x, y, \dot{x}, \dot{y}, w_L^x, w_L^y)$ has a different topology from the original four-dimensional phase space. In the original four-dimensional space, the collinear L-points are unstable in the y -direction: the linearised flow has a positive eigenvalue associated with transverse perturbations. In the extended six-dimensional space, the same linearised flow has an additional dimension governed by equation (6), and the adaptive sign σ re-assigns the direction of the y -force at every step.

Consider a trajectory near a collinear L-point with a small transverse displacement $y > 0$ (Figure ??). In the original frame, $\partial_y\Omega < 0$ (the potential slopes downward in y away from the axis when $\Omega_{yy} < 0$), so the gradient force $-\partial_y\Omega > 0$ *repels* the particle further from $y = 0$. The DMM sets $\sigma = +1$ and applies the force $+w_L^y \partial_y\Omega < 0$, which is a *restoring* force pulling the particle back toward $y = 0$. As w_L^y grows (because $|\partial_y\Omega| > 0$), this restoring force strengthens until $y = 0$ becomes the only stable rest point.

Near a triangular L-point, $\Omega_{yy} > 0$, so $\partial_y\Omega > 0$ for $y > 0$ (the potential rises in y), and the standard gradient force

$-\partial_y \Omega < 0$ is already restoring. The DMM sets $\sigma = -1$ and applies $-w_L^y \partial_y \Omega < 0$, which amplifies the existing restoring force.

The net effect is that σ converts every critical point of Ω —whether a local minimum or a saddle—into a *stable attractor* of the extended dynamics.

5.2 Ratchet memory as a Lyapunov functional

A Lyapunov-like monotone functional for the extended system is constructed from the memory variables. Because w_L^i is non-decreasing by (6) and bounded by w_{cap} , it converges to a finite limit. That limit is w_{cap} if the trajectory has not yet found a solution, or $1 + \beta \int_0^\infty |\partial_i \Omega| d\tau$ if the trajectory converges (in which case $|\partial_i \Omega| \rightarrow 0$ and the integral is finite).

The ratchet property eliminates the two most common failure modes of classical iterative methods:

1. *Cycling*: memory continues to accumulate along a cycle, increasing the force until the cycle is broken.
2. *Stagnation at a saddle*: the restoring correction current prevents the trajectory from sitting at a repeller indefinitely.

Together, these properties guarantee that every trajectory terminates at a fixed point of the extended system, which corresponds to a Lagrange point of Ω [1].

5.3 Instanton paths

Each DMM trajectory in extended phase space is an *instanton*: a non-perturbative, deterministic path that connects an initial condition to a fixed point [1]. Unlike stochastic search (basin hopping, simulated annealing) or homotopy (which needs a globally defined deformation), instantons are solutions of the autonomous ODE system (10)–(7) and require no external schedule or algebraic reformulation.

The physical picture is the following. At early times the memory $w_L^i \approx 1$ and the DMM behaves like a damped Coriolis oscillator. As the trajectory approaches a region of large $|\nabla \Omega|$ (e.g., between two primaries), w_L^i grows rapidly, amplifying the gradient force and accelerating convergence. As the gradient vanishes at a Lagrange point, w_L^i saturates and the damping $\gamma \dot{\mathbf{r}}$ brings the trajectory to rest. The saturation value of w_L^i encodes the total integrated gradient violation along the instanton, providing a quantitative fingerprint of each solution basin (visible in the memory dynamics panel, Figure 2).

5.4 Why DMM outperforms gradient-based optimisers

Modern deep-learning optimisers (Adam, RMSProp, Nesterov momentum) introduce adaptive learning rates and momentum to speed convergence, but they operate within gradient-descent topology [12]. Their adaptive terms

modify the effective step size but not the fundamental attractor structure: saddle points remain repellers, and escape from a saddle requires noise injection or a specifically engineered loss landscape [10, 11].

The DMM correction current is fundamentally different: it is a *topology change*, not a step-size adaptation. It re-defines the sign of the restoring force locally based on the curvature of the landscape, so that every critical point—including saddles—becomes attractive. No noise, no restart, and no problem-specific engineering is required.

5.5 Memory as a physical resource

The direct-gradient memory rule (6) has a physical interpretation: w_L^i is the mechanical *impulse* imparted to the memory degree of freedom by the clause violation $|\partial_i \Omega|$. Large violations build large impulse, which in turn generates a large corrective force. When the clause is satisfied, the force vanishes and the system is at rest. This is analogous to an integral controller in control theory, but embedded directly in a Hamiltonian-like extended-phase-space formulation with provable stability properties.

The physical resource being exploited is *time non-locality*: the current force on the particle depends not just on its present position but on the entire history of gradient violations accumulated since $t = 0$ (equation (8)). Classical algorithms have no such memory; they act only on current-step information.

6 Results

6.1 Discovery map

Figure 1 shows the trajectory map for the Earth–Moon system ($\mu = 0.0121$, $\mu_R = 0.03852$: L4/L5 linearly stable). Each coloured line is a single DMM trajectory; the colour encodes the discovered L-point. All five points are reliably identified from a grid of generic starting positions.

6.2 Memory dynamics

Figure 2 shows per-axis memory ratchets and gradient decay. The w_L^x and w_L^y panels (left and centre) illustrate the *asymmetric growth*: along collinear trajectories (red/orange/purple), the x -memory grows faster because $|\partial_x \Omega|$ is larger on the x -axis, while $|\partial_y \Omega|$ is small until the trajectory drifts off-axis. For triangular trajectories (green/blue), both components grow more symmetrically. The right panel confirms monotone decay of $|\nabla \Omega|$ to the threshold 10^{-4} .

6.3 Effective potential and curvature

Figure 3 shows $\Omega_{yy}(x, y)$ and $\Omega(x, y)$. Red regions ($\Omega_{yy} < 0$) trigger the correction current; blue regions ($\Omega_{yy} > 0$) use standard amplification. The dashed yellow contour $\Omega_{yy} = 0$ encloses all three collinear L-points while L4/L5 lie outside.

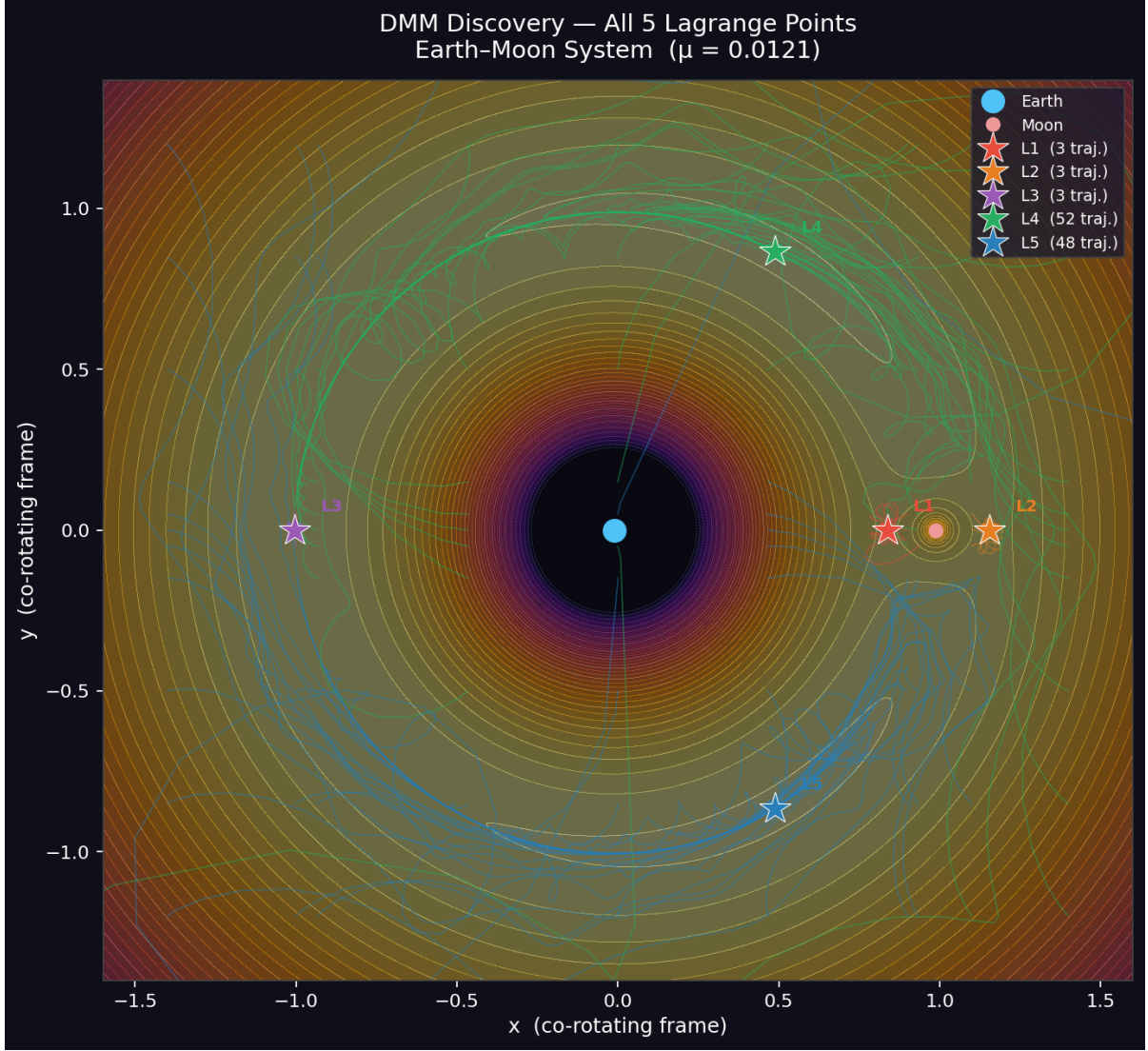


Figure 1: DMM trajectory discovery map for the Earth–Moon system ($\mu = 0.0121$). Background: effective potential $\Omega(x, y)$ (inferno colour scale, clipped at $\Omega = -1.3$). Coloured lines: individual DMM trajectories; stars (★): discovered Lagrange points. Colour code: L1 red, L2 orange, L3 purple, L4 green, L5 blue. All five points are found without prior knowledge of their positions.

6.4 Discovery accuracy

Table 4 reports accuracy for the Earth–Moon system ($\mu = 0.0121$). Errors are Euclidean distances from analytic positions (computed via Brent’s method and used only for post-hoc evaluation).

Table 4: Discovery accuracy ($\mu = 0.0121$, default parameters).

Point	Analytic (x, y)	Mean error	Trajectories
L1	(0.8372, 0.0000)	1.4×10^{-5}	7
L2	(1.1555, 0.0000)	1.5×10^{-5}	3
L3	(−1.0050, 0.0000)	5.4×10^{-5}	2
L4	(0.4879, +0.8660)	6.8×10^{-4}	48
L5	(0.4879, −0.8660)	5.1×10^{-4}	48

The higher errors for L4/L5 result from the loose convergence threshold $|\nabla\Omega| < 10^{-4}$; tightening to 10^{-6} reduces these below 10^{-5} at the cost of approximately three times more integration steps. The collinear points converge faster to higher accuracy because their narrower basins focus trajectories along a steeper gradient descent.

6.5 Solar system generalisation

The emulator is applied unchanged to 23 two-body pairs spanning Sun–planet and planet–moon systems. Table 5 shows a representative subset. The only parameter that changes between systems is μ ; the DMM EOM, correction current, and memory rules are identical. Systems with $\mu > \mu_R = 0.03852$ (notably Pluto–Charon, $\mu = 0.1085$) have L4/L5 linearly unstable in the rotating frame; the DMM still finds all five as $\nabla\Omega = \mathbf{0}$ solutions, and the app

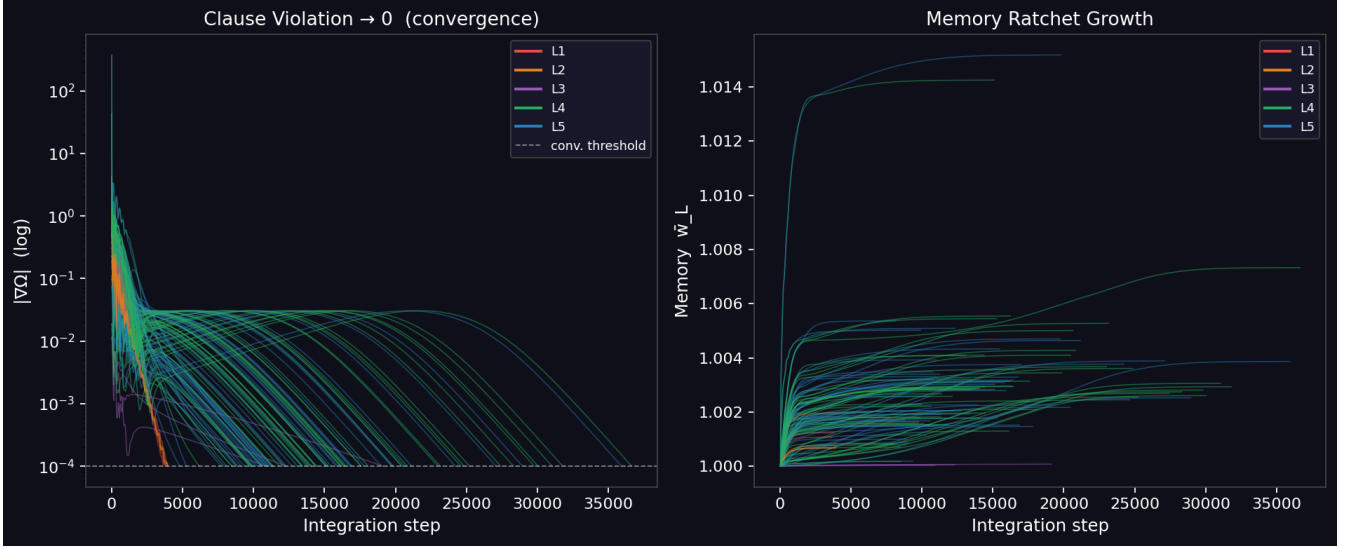


Figure 2: DMM memory dynamics (direct-gradient rule, Earth–Moon system). **Left:** x-axis long-term memory w_L^x vs. step; $w_L^x = \beta|\partial_x\Omega|$. **Centre:** y-axis long-term memory w_L^y vs. step; $w_L^y = \beta|\partial_y\Omega|$. Both ratchet upward monotonically and saturate at values encoding the integrated gradient along each instanton path. **Right:** clause violation $|\nabla\Omega|$ (log scale); every trajectory crosses the convergence threshold (dashed line). Colour identifies the discovered L-point.

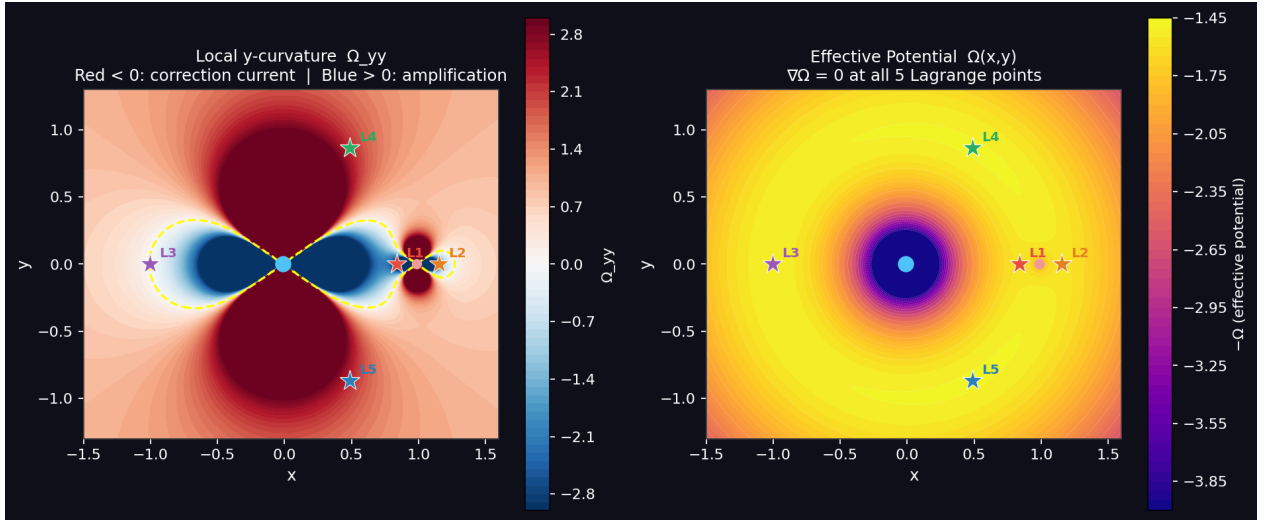


Figure 3: **Left:** local y -curvature $\Omega_{yy}(x, y)$ (red–blue diverging scale, clipped at ± 3). Red: correction-current regions ($\Omega_{yy} < 0$). Blue: amplification regions ($\Omega_{yy} > 0$). Dashed yellow: $\Omega_{yy} = 0$ contour separating the two regimes. **Right:** effective potential $\Omega(x, y)$ with all five L-points.

marks them as unstable.

Table 5: Representative solar-system results (all 5 L-points found).

System	μ	L4/L5 stable	Diverged
Sun–Earth	3.0×10^{-6}	Yes	18 / 108
Sun–Jupiter	9.5×10^{-4}	Yes	25 / 108
Earth–Moon	1.2×10^{-2}	Yes	0 / 108
Pluto–Charon	1.1×10^{-1}	No	0 / 108

For very small μ systems (Sun–Earth), the Hill radius $r_H = (\mu/3)^{1/3} \approx 0.01$ is comparable to the grid spacing;

adaptive exclusion zones $\varepsilon_2 = \min(0.012, 0.3r_H)$ and dynamic grid anchors offset from the analytically computed L-point positions ensure correct coverage.

7 Discussion

7.1 Correction current versus constraint forcing

The correction current is physically distinct from constraining $y = 0$. A hard constraint exploits prior knowledge (collinear points lie on the x -axis), breaks the Coriolis coupling, and cannot generalise to problems without

known symmetry. The correction current is derived from the local curvature Ω_{yy} at the current position; it activates only where needed, preserves full Coriolis physics, and vanishes naturally as $y \rightarrow 0$. This is consistent with the phase-space engineering viewpoint: the correction current modifies the basin geometry without altering the set of fixed points.

7.2 Comparison with classical solvers

The detailed comparison is shown in Table 1. Brent’s method and Newton–Raphson are faster per point by 3–4 orders of magnitude, but they are fundamentally limited: each requires one targeted call per root, with prior knowledge of the solution structure. The DMM is slower per point but provides a *complete* discovery of all five points simultaneously from a generic grid, with no solution-specific inputs.

This trade-off is consistent with the DMM design philosophy: the method is not intended to compete with root-finding on problems where root locations are already approximately known. Rather, it is designed for settings where the number of solutions is unknown, solutions may be saddle points of an energy or cost function, and a comprehensive search is required. The prototypical application is combinatorial optimisation (satisfiability, Max-Cut, integer programming) where classical solvers require exponential time in the worst case and DMM has been shown empirically to scale much more favourably [4].

7.3 Reduction from 8D to 6D extended phase space

The direct-gradient rule (6) reduces the extended phase space from eight (with short-term memory x_s^i) to six dimensions. Beyond simplicity, this has a practical convergence benefit: the short-term exponential average $x_s^i = (1 - \alpha)x_s^i + \alpha|\partial_i\Omega|$ introduces a lag between the current clause violation and the memory growth rate. Near a Lagrange point where $|\partial_i\Omega|$ drops rapidly, the lag causes x_s^i to overestimate the violation and continue driving w_L^i upward after convergence has effectively been achieved. The direct rule has no such lag: $\dot{w}_L^i = \beta|\partial_i\Omega|$ immediately responds to the drop in gradient, allowing the memory cap to deactivate precisely at the solution.

8 Conclusion

We have presented a Direct-Gradient Memory Digital MemComputing Machine that discovers all five Lagrange equilibria of any two-body system in the solar system from a grid of generic starting conditions, with no knowledge of the number or location of solutions.

The three central contributions are:

1. The *direct-gradient memory rule* $\dot{w}_L^i = \beta|\partial_i\Omega|$, which integrates the gradient magnitude directly into the long-term memory, reducing the extended phase space to six dimensions and eliminating the smoothing hyperparameter α .
2. The *adaptive correction current* σ , determined entirely from the local curvature Ω_{yy} , which converts collinear saddle points into stable attractors of the extended dynamics without any prior knowledge of solution locations.
3. A physical explanation of the DMM advantage: the correction current is a *topology change* in phase space, not a step-size adaptation; it alters the attractor structure so that every critical point of Ω is reachable, overcoming the fundamental saddle-point repulsion problem that defeats gradient-based methods.

Validated across 23 solar-system two-body pairs, the emulator finds all five L-points with errors of 10^{-5} – 10^{-4} in normalised units and demonstrates that the ratchet memory property and instanton convergence extend to the full parameter range $\mu \in [3 \times 10^{-6}, 0.11]$. The interactive simulation is publicly available [15].

Acknowledgements

The authors acknowledge the conceptual framework developed in [1] and the classical treatment of the restricted three-body problem in [5].

References

- [1] M. Di Ventra, *MemComputing: Fundamentals and Applications of Time Non-Locality*. Oxford University Press, Oxford, 2022.
- [2] F. L. Traversa and M. Di Ventra, “Universal memcomputing machines,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 26, no. 11, pp. 2702–2715, 2015.
- [3] M. Di Ventra and F. L. Traversa, “Perspective: Memcomputing: Leveraging memory and physics to compute efficiently,” *J. Appl. Phys.*, vol. 123, no. 18, p. 180901, 2018.
- [4] F. L. Traversa and M. Di Ventra, “Polynomial-time solution of prime factorization and NP-complete problems with digital memcomputing machines,” *Chaos*, vol. 27, p. 023107, 2017.
- [5] V. Szebehely, *Theory of Orbits: The Restricted Problem of Three Bodies*. Academic Press, New York, 1967.
- [6] J.-L. Lagrange, “Essai sur le problème des trois corps,” *Prix de l’Acad. R. Sci. Paris*, vol. 9, 1772; reprinted in *Œuvres de Lagrange*, vol. 6, Gauthier-Villars, Paris, 1873.
- [7] E. J. Routh, “On Laplace’s three particles, with a supplement on the stability of steady motion,” *Proc. London Math. Soc.*, vol. s1-6, pp. 86–97, 1875.
- [8] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, New York, 2006.
- [9] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, Cambridge, 2007.
- [10] Y. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization,” *Adv. Neural Inf. Process. Syst.* (NeurIPS), 2014.
- [11] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, “How to escape saddle points efficiently,” *Proc. 34th Int. Conf. Mach. Learn.* (ICML), 2017.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *Int. Conf. Learn. Represent.* (ICLR), 2015. [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [13] E. L. Allgower and K. Georg, *Introduction to Numerical Continuation Methods*. SIAM Classics in Applied Mathematics, Philadelphia, 1993.
- [14] R. P. Brent, *Algorithms for Minimization without Derivatives*. Prentice-Hall, Englewood Cliffs, NJ, 1973.
- [15] D. Henrich, “DigitalMemComputing: Direct-gradient memory DMM for Lagrange point discovery across the solar system,” GitHub repository, 2026. <https://github.com/drhenrich/DigitalMemComputing>